# SIGA UEA · Arquitetura em camadas

HT03 (SCRUM-108) · Arquitetura e contrato de API · versão de 20/10/2026, sobre o código da branch `dev`.

Este documento descreve como o sistema está organizado em camadas, o caminho de uma requisição e as decisões que sustentam essa organização. Ele acompanha os diagramas de componentes e de pacotes (`SIGA_UEA_componentes_e_pacotes.drawio`), o diagrama de implantação (`docs/modelagem/diagrama-implantacao-siga-uea.drawio`) e o contrato da API (`docs/arquitetura/contrato-api-siga-uea.html`).

## 1. Visão geral

O SIGA UEA é uma aplicação web em três partes, que conversam só por contratos explícitos:

| Parte | Tecnologia | Papel |
|---|---|---|
| **Site** | Next.js 16 com React 19 (TypeScript, Tailwind) | Telas e navegação. Não guarda regra de negócio nem acessa o banco. |
| **API** | Python 3.12 com FastAPI, SQLAlchemy 2.1 e Alembic | Toda regra de negócio, autorização, validação e auditoria. Exposta como REST/JSON sob `/api/v1`. |
| **Banco** | PostgreSQL 16 (SQLite no desenvolvimento) | Dados, restrições de integridade e o log de auditoria imutável. Esquema criado por migrações. |

O site e a API integram-se **somente pelo contrato da API**; a API e o banco, **somente pelo esquema versionado nas migrações** (Plano de Integração, 4.6.1). O e-mail fica encapsulado num componente próprio (`services/email.py`), que pode ser trocado sem mexer no resto.

É um **monólito modular**: uma única API, organizada por módulo de negócio (usuários, turmas, cadastro de TCC; depois defesa, publicação, agenda), e não microsserviços. Para cinco pessoas com 8 h por semana, um só serviço para publicar, testar e manter reduz o risco de integração, que é o principal risco técnico do projeto (Plano de Integração, seção 2).

```
Navegador ──HTTPS──▶ Site (Next.js) ──/api/v1 (rewrite)──▶ API (FastAPI) ──SQL/TLS──▶ PostgreSQL
                                                                  └──SMTP──▶ Servidor de e-mail
```

## 2. Camadas do site (`app/`, `components/`, `lib/`, `proxy.ts`)

| Camada | Onde | Responsabilidade |
|---|---|---|
| **Páginas** | `app/(publico)/` (entrar, ativar, esqueci-senha, redefinir-senha · TEL01) e `app/(sistema)/` (início, usuários e perfis, meu perfil, TCCs, cadastro, acompanhamento, orientações, turmas) | Uma pasta por tela. As telas internas são componentes de cliente ( `"use client"` ) que chamam a API e mostram o resultado. |
| **Componentes de interface** | `components/` (campo, botão, modal, aviso, casca, selos de situação, cartão de turma) | Peças reutilizadas, sem acesso à API. Garantem o mesmo visual e a acessibilidade em todas as telas. |
| **Cliente da API e sessão** | `lib/api.ts` , `lib/sessao.tsx` , `lib/sessao-servidor.ts` , `lib/tipos.ts` | `api.ts` é o único ponto que faz chamadas HTTP: monta a requisição, lê o erro no formato padrão e leva ao login quando a API responde 401. `sessao-servidor.ts` pergunta à API "quem sou eu?" antes de desenhar uma tela interna; `sessao.tsx` entrega o usuário logado às telas. `tipos.ts` espelha os esquemas da API. |
| **Proxy de navegação e repasse** | `proxy.ts` e `next.config.ts` (rewrites) | `proxy.ts` manda para o login quem abre uma tela interna sem o cookie de sessão; é só um atalho, porque quem valida a sessão é sempre a API. Os rewrites repassam `/api/v1/*` para a API, de modo que o navegador fala com uma única origem. |

**Regra:** o site nunca decide permissão. Esconder um item do menu é conveniência; a autorização de verdade é conferida pela API em toda rota.

## 3. Camadas da API ( `backend/app/` )

```
routes/  ──▶  services/  ──▶  models.py  ──▶  banco (database.py, migrations/)
   │             │
schemas/      auditoria, e-mail, tokens          transversais: deps.py, security.py, erros.py, config.
```

| Camada | Onde | Responsabilidade | Não faz |
|---|---|---|---|
| **Rotas** | `routes/` (auth, perfil, usuarios, areas, catalogo, turmas, tccs, health) | Recebe a requisição HTTP, declara o perfil exigido, converte a entrada pelo esquema e chama um serviço. | Não tem regra de negócio nem consulta o banco diretamente. |
| **Esquemas** | `schemas/` (Pydantic) | Formato de entrada e de saída de cada rota: tipos, obrigatoriedade, tamanhos e validações de campo (ex.: resumo de até 500 palavras, até 10 palavras-chave). Entrada inválida vira **400** com a mensagem de cada campo. | Não acessa o banco. |
| **Segurança e sessão** | `deps.py`, `security.py` | `usuario_atual` exige sessão válida e renova os 60 minutos; `exigir_perfil(...)` devolve 403 a quem não tem o perfil. `security.py` aplica a regra de senha, gera o hash bcrypt (custo ≥ 10) e os tokens de link. | Não conhece telas. |
| **Serviços** | `services/` (autenticacao, tokens, usuarios, perfil, areas, turmas, tccs, auditoria, email) | **Regras de negócio**: RN04 (um TCC por período), RN05 (matrícula), RN07 (vínculo), elegibilidade do orientador, geração do protocolo, transições de situação. Cada operação é uma transação; cada alteração grava no log de auditoria. Os comentários citam o passo do diagrama de sequência (DS-FL01, DS-FL03…). | Não monta resposta HTTP: sinaliza erro com `ErroAPI(código, mensagem)`, que a camada de rotas devolve. |
| **Modelos** | `models.py` (SQLAlchemy) | Tabelas, relacionamentos e restrições (únicas, CHECK de domínio). Os nomes seguem o modelo lógico (`docs/modelagem`). | Não tem regra de negócio. |
| **Acesso ao banco e esquema** | `database.py`, `migracoes.py`, `migrations/` (Alembic), `seed.py` | Uma sessão por requisição, fechada assim que a rota termina; migrações versionadas 0001 a 0004; carga inicial repetível. | — |

**Transversais:** `erros.py` dá o formato único de erro (`detail`, `campos`, `campo`, `criterios`, `motivo`, `campos_pendentes`); `config.py` lê as variáveis de ambiente e o `backend/.env`; `services/auditoria.py` grava o log que o banco não deixa alterar (RNF03); `services/email.py` registra cada envio em `notificacao_email` e envia **depois** da resposta, sem atrasar quem pediu.

A pasta `repositories/` existe, mas está vazia: os serviços usam o ORM diretamente. Ela fica reservada caso alguma consulta cresça a ponto de merecer uma camada própria.

## 4. Caminho de uma requisição: envio do cadastro de TCC

Exemplo de `POST /api/v1/tccs/{id}/submissao` (DS-FL01, RBN03.03 e RBN03.04):

1. **Navegador → site:** o aluno confirma "Enviar para análise" em TEL05. `lib/api.ts` faz `POST /api/v1/tccs/12/submissao` no mesmo endereço do site, levando o cookie `siga_sessao`.
2. **Site → API:** o rewrite do Next.js repassa a chamada para a API.

3. **Rota** ( `routes/tccs.py` ): a dependência `exigir_perfil(aluno)` confere a sessão (401 se inválida) e o perfil (403); a rota chama `tccs.submeter` .
4. **Serviço** ( `services/tccs.py` ):
5. carrega o TCC **travando a linha** ( `SELECT … FOR UPDATE` ), para que dois cliques não gerem dois protocolos (RNF06);
6. confere o dono (404 se for de outro aluno, RN07), a matrícula (403, RN05) e as pendências (422 com `campos_pendentes` , por exemplo o orientador ainda não aceitou);
7. se já estava em análise, devolve o mesmo protocolo;
8. se não, gera `TCC-AAAA-NNNNN` , muda a situação para EM_ANALISE, grava o histórico e o log de auditoria na mesma transação e agenda o e-mail ao orientador.
9. **Resposta:** 200 com o protocolo. A sessão do banco fecha; o e-mail sai em seguida, com conexão própria.
10. **Site:** a tela mostra o protocolo e bloqueia a edição.

Esse caminho foi medido com 30 envios simultâneos: p95 de 0,73 s, contra a meta de 3 s do RNF01 (relatório de execução, aba "Desempenho RNF01").

## 5. Segurança (RNF02)

| Ameaça ou requisito | Como a arquitetura responde |
|---|---|
| Roubo de sessão por script na página | Token só em cookie `httponly` (o JavaScript não lê), `SameSite=Lax` , `Secure` em produção ( `COOKIE_SEGURO=1` ). |
| Senha vazada do banco | Só o hash bcrypt (custo ≥ 10) fica gravado; tokens de link e de sessão também só como hash. |
| Força bruta no login | Bloqueio de 15 minutos após 3 tentativas; mensagem genérica que não revela se o e-mail existe. |
| Acesso a dado de outro usuário | Autorização por perfil em toda rota e por vínculo no serviço (RN07); sem vínculo, a API responde 404, sem revelar que o registro existe. |
| Interceptação | HTTPS em todo o caminho; TLS também entre a API e o banco no ambiente integrado. |
| Alteração do histórico | Log de auditoria com trigger no banco que recusa UPDATE e DELETE (migração 0003, RNF03). |

## 6. Dados e consistência

- **Integridade no banco, não só no código:** e-mail único sem diferenciar maiúsculas ( `lower(email)` ), um TCC por aluno e período ( `uq_tcc_aluno_periodo` ), protocolo único e domínios de situação com CHECK.
- **Transações curtas:** cada operação do serviço é uma transação. A sessão fecha quando a rota termina, antes das tarefas em segundo plano; sem isso, sob carga, cada requisição segurava duas conexões e o pool se esgotava (corrigido na S03 e medido no teste de carga).
- **Esquema só por migração:** o mesmo esquema no SQLite de desenvolvimento e no PostgreSQL. Um teste falha se alguém mudar `models.py` sem criar a migração.

## 7. Ambientes

| Ambiente | Onde roda | Uso |
|---|---|---|
| Desenvolvimento | `npm run dev` + `uvicorn --reload` + SQLite, ou Docker Compose | Trabalho do dia a dia (README, "Como rodar"). |
| Integrado | Vercel (site), Render (API), Neon (PostgreSQL) | Homologação e entregas (HT06). Configuração em `render.yaml`, `.vercelignore` e README, "Publicar na internet". |

Detalhes, portas e protocolos estão no diagrama de implantação.

## 8. Decisões de arquitetura

| Decisão | Motivo | Alternativa descartada |
|---|---|---|
| Monólito modular (uma API por módulos) | Equipe pequena, integração como maior risco; um só serviço para publicar e testar | Microsserviços por módulo |
| Site e API na mesma origem, com repasse de `/api/v1` | Cookie de sessão sem CORS e sem expor a API diretamente | Site chamando a API em outro domínio |
| Sessão em cookie `httponly` | Protege o token de scripts na página | Token em `localStorage` |
| Regras no serviço, rotas finas | Regras testáveis sem HTTP e reaproveitáveis entre rotas | Regra dentro da rota ou na tela |
| Validação responde 400, regra de negócio 409/422 | É o que as fichas pedem; separa "campo inválido" de "operação não permitida" | O 422 padrão do FastAPI para tudo |
| Esquema por migrações (Alembic) | Mesmo banco em todos os ambientes, histórico versionado no Git (HT05) | `create_all` no início da aplicação |
| E-mail depois da resposta, com registro em `notificacao_email` | A falha de envio não desfaz o cadastro e pode ser reenviada (RBN01.05, RBN03.06) | Enviar dentro da requisição |
| Restrições no banco além do código | Duplicidade e concorrência barradas mesmo com dois servidores ou dois cliques (RNF06) | Conferir só no código |

## 9. Como a arquitetura é verificada

- **Suíte da API** (`backend/tests`, pytest): 122 execuções, em SQLite e PostgreSQL 16, uma por critério de aceite; inclui as 51 operações do contrato.
- **Testes no navegador** (`docs/testes/e2e`): fluxos completos pela tela, responsividade em 360 e 768 px.
- **Teste de carga** (`backend/scripts/carga_rnf01.py`): RNF01 e RNF06.
- Resultados e evidências: `docs/testes/Relatorio_Execucao_Testes_SIGA_UEA.xlsx`.